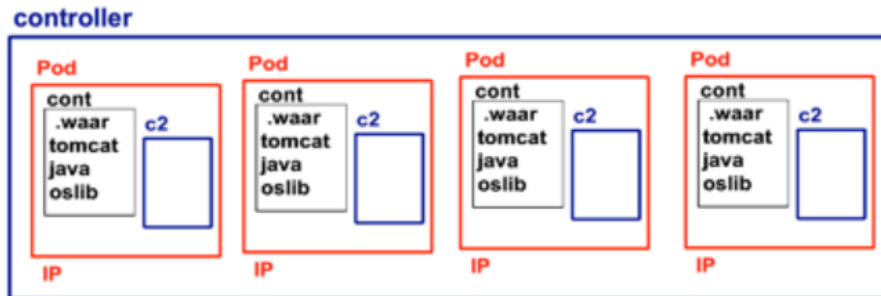


UC: If we delete the Pod then Pod is deleted, we have to recreate manually. I want to run multiple replicas, and want to maintain a desired state.

UC: How to create multiple Pods in a single command/ How we run more than one pod to achieve HA?

Controllers in K8s: logical group of pods from same image, controls set of Pods from a single command, also provides the extra sets of features.

Is an abstraction layer that can create and manage a single/group of pods.



- Scaling
- Replicas- Number of Pods
- Auto healing (desired vs. current)

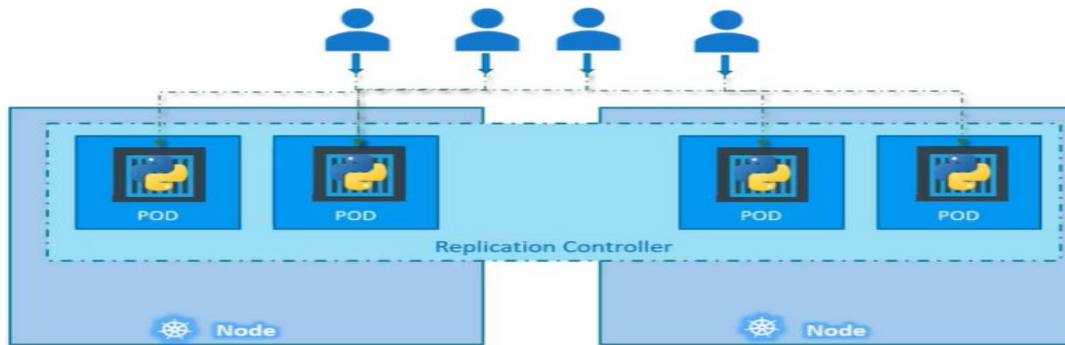
Types of Controllers?

- ReplicationController: (Equity based selector)
- ReplicaSet (Set based selector)
- Deployment (version control, zero downtime, RollingUpdate)
- DaemonSet
- Job
- CronJob
- StatefulSet

Pods controls the container; Controller controls the multiple pods together. Sits on the top of Pods

Replication Controller (Equity based)

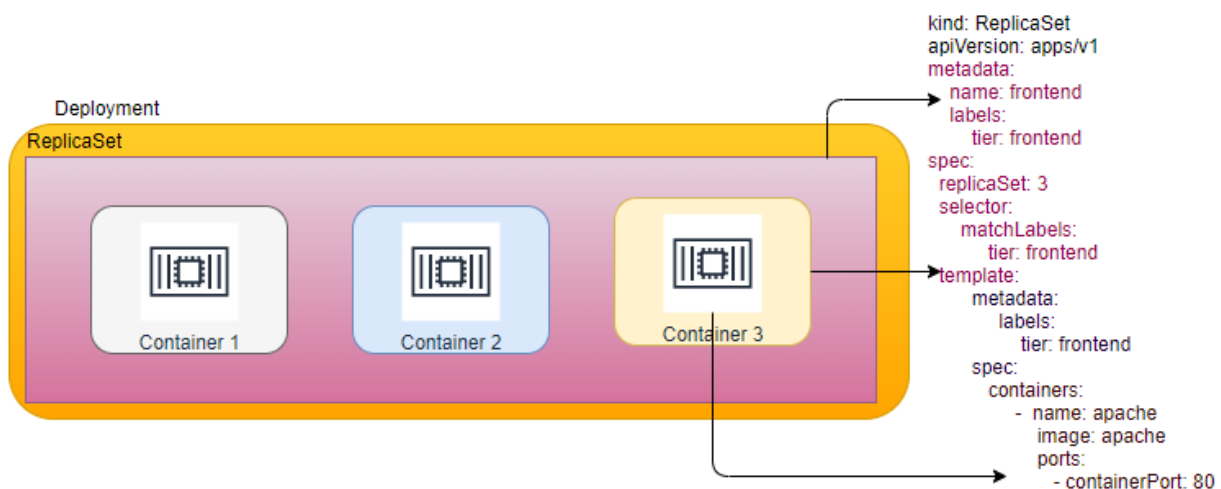




Do we need any controller if we are plan for single Pod?

Yes, controller helps us to bringing up a new Pod In case of existing Pod fails.

ReplicaSet (Set based selector)



ReplicaSet

- It allows you to create the requested number of pods.
- It allows to scale up/down.
- It provides desired state.
- Ensures that a specified number of Pods are running at any time.
- If there are excess pods, they get killed and vice versa.
- New pods are launched when they get fail, get deleted or terminated.

Replication Set and Pods are associated with 'labels and selector', creating an 'rs' with count of 1 ensure that a pod is always available.

What is difference between Replication Controller and ReplicaSet?

ReplicaSet is next-generation Replication Controller.

Replicaset can **also manage the Pods which are not created as definition of rs** if number of pods already exist then it will not create any pod. (selector is not a mandatory field in case of ReplicationController)

```
...
selector:
  app: nginx
  tier: frontend
...
```

=

```
...
selector:
  matchLabels:
    app: nginx
    tier: frontend
...
```

Supports on Older Resources such as:

- ReplicationControllers,
- Services

Supports on newer resources such as:

- ReplicaSets
- Deployments
- Jobs
- DaemonSet

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
    app: myapp
    type: front-end
spec:
  template:
    

POD


```

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
  replicas: 3
```

```
apiVersion: v1
kind: Pod
```

```
> kubectl create -f rc-definition.yml
replicationcontroller "myapp-rc" created

> kubectl get replicationcontroller
```

NAME	DESIRED	CURRENT	READY	AGE
myapp-rc	3	3	3	19s

```
> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-rc-41vk9	1/1	Running	0	20s
myapp-rc-mc2mf	1/1	Running	0	20s
myapp-rc-px9pz	1/1	Running	0	20s

ReplicaSet



replicaset-definition.yaml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: simple-webapp
  labels:
    app: App1
    function: Front-end
spec:
  replicas: 3
  selector:
    matchLabels:
      app: App1
  template:
    metadata:
      labels:
        app: App1
        function: Front-end
    spec:
      containers:
        - name: simple-webapp
          image: simple-webapp
```

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

```
apiVersion: v1
kind: Pod
```

```
> kubectl create -f replicaset-definition.yaml
```

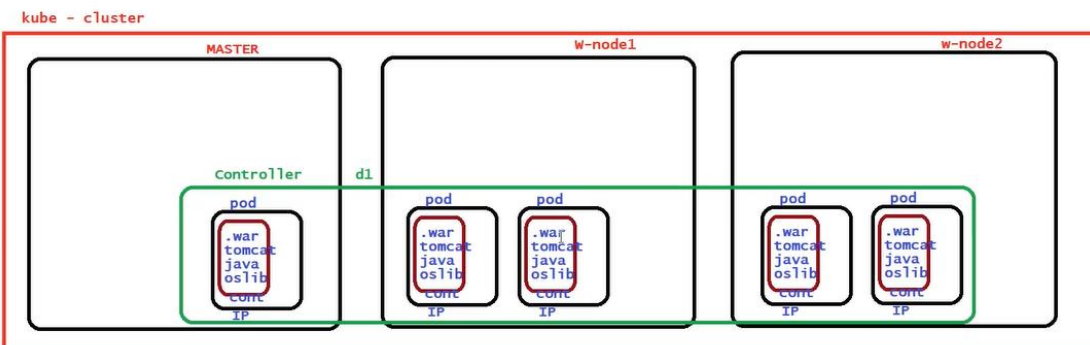
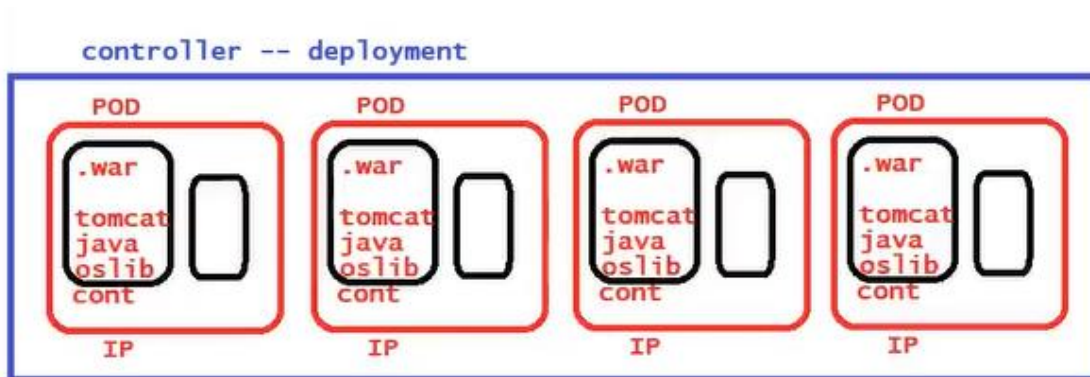
```
replicaset "myapp-replicaset" deleted
```

```
> kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
myapp-replicaset	3	3	3	19s

Deployment (rolling update)

- It allows you to create the requested number of pods.
- It allows to scale up/down.
- It provides desired state.
- rolling update/back
 - we can perform no downtime deployment
 - we can upgrade from v1 to v2 without giving downtime



Definition

```
> kubectl create -f deployment-definition.yml
```

```
deployment "myapp-deployment" created
```

```
> kubectl get deployments
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
myapp-deployment	3	3	3	3	21s

```
> kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
myapp-deployment-6795844b58	3	3	3	2m

```
> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-deployment-6795844b58-Srbj1	1/1	Running	0	2m
myapp-deployment-6795844b58-h4w55	1/1	Running	0	2m
myapp-deployment-6795844b58-lfjhw	1/1	Running	0	2m

deployment-definition.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

Deployment: Wrapper over ReplicaSet also provides RollingUpdate

Use Case: Imagine, you are upgrading application from v1 to v2

- Upgrade with zero downtime?
- Upgrade sequentially one after other?
- Pause and resume upgrade process?
- Rollback upgrade to previous stable release?

Features

- Multiple Replicas
- Upgrade
- Rollbacks'
- Scale up or down
- Pause and Resume

Deployments Types

- Recreate: downtime while switching from v1 to V2
- RollingUpdate (Ramped or Incremental): Default
- Canary: Two deployments with same selector
- Blue(V1)/Green (V2): Update the selector in service

```

kind: Deployment
apiVersion: apps/v1
metadata:
  name: d1
spec:
  replicas: 4
  selector:
    matchLabels:
      role: app
  template: ## pod definition which include pod metadata & spec
    metadata:
      labels:
        role: app
    spec:
      containers:
        - name: cont1
          image: lerndevops/samplepyapp:v1
          ports:
            - name: http
              containerPort: 3000

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mydeployment
spec:
  replicas: 5
  selector:
    matchLabels:
      app: myapp
  minReadySeconds: 20 # wait for 10 sec before pod is ready going to next
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1 # take down 1 pod at a time
      maxSurge: 1 # bring one at a time
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp-cont
          image: lerndevops/samplepyapp:v2
          ports:
            - containerPort: 3000

```

kc deploy nginx-deploy --image=nginx --port=80 --replicas=5 --dry-run=client -o yaml> nginx-deploy. yaml

kind: Deployment # k api-resources
apiVersion: apps/v1 # k explain < resource>

Rollout and Versions:

UC: For a minute, let us forget about PODs and replicaset and other kubernetes concepts and talk about how you might want to deploy your application in a production environment. Say for example you have a **webserver** that needs to be deployed in a production environment. You need not ONE, but many such instances of the webserver running for obvious reasons.

Secondly, when newer versions of application builds become available on the docker registry, you would like to UPGRADE your docker instances seamlessly.

However, when you upgrade your instances, you do not want to **upgrade all of them at once** as we just did. This may impact users accessing our applications, so you may want to **upgrade them one after the other**, and that kind of upgrade is known as Rolling Updates. Suppose one of the upgrades you performed resulted in an unexpected error and you are asked to undo the recent update. You would like to be able to roll back the changes that were recently carried out.

Finally, say for example you would like to make multiple changes to your environment such as upgrading the underlying Webserver versions, as well as scaling your environment and also modifying the resource allocations etc. **You do not want to apply each change immediately after the command is run, instead you would like to apply a pause to your environment, make the changes and then resume so that all changes are rolled-out together.**

All of these capabilities are available with the kubernetes Deployments. As of now we discussed about PODs, which deploy single instances of our application such as the web application in this case. Each container is encapsulated in PODs. Multiple such PODs are deployed using Replication Controllers or Replica Sets, and then comes **Deployment** which is a kubernetes object that comes higher in the hierarchy. **The deployment provides us with capabilities to upgrade the underlying instances seamlessly using rolling updates, undo changes, and pause and resume changes to deployments.**

Before we look at how we upgrade our application, let's try to understand Rollouts and Versioning in a deployment.



Whenever you create a new deployment or upgrade the images in an existing deployment it triggers a Rollout. A rollout is the process of gradually deploying or upgrading your application containers. When you first create a deployment, it

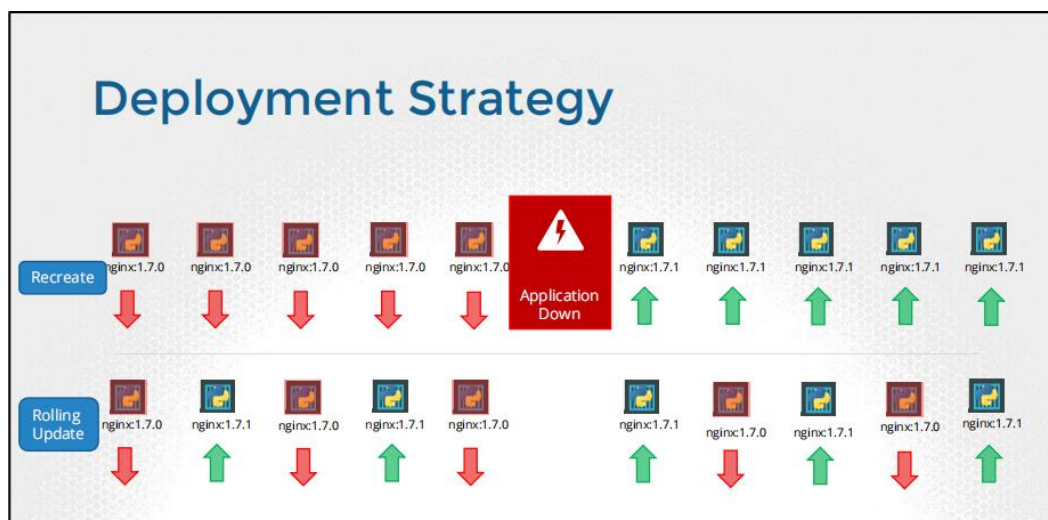
triggers a rollout. A new rollout creates a new Deployment revision. Let's call it revision 1. In the future when the application is upgraded – meaning when the container version is updated to a new one – a new rollout is triggered and a new deployment revision is created named Revision 2. This helps us keep track of the changes made to our deployment and enables us to roll back to a previous version of deployment if necessary.

You can see the status of your rollout by running the command: **kubectl rollout status** followed by the name of the deployment. To see the revisions and history of rollout run the command **kubectl rollout history** followed by the deployment name and this will show you the revisions.

```
kubectl rollout status deploy <name>
kubectl rollout history deploy <name>
kubectl rollout undo deploy <name>
kubectl rollout pause deploy <name>
kubectl rollout resume deploy <name>
kubectl rollout restart deploy <NAME>
```

Deployment Strategies- No downtime

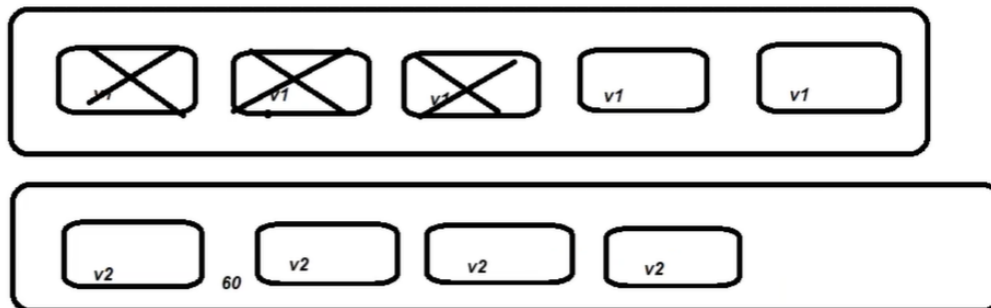
- RollingUpdate
- blue v1-green v2
- canary



What is RollingUpdate is the default deployment strategy?

A rolling deployment slowly replaces instances of the previous version of an application with instances of the new version of the application. A rolling deployment typically waits for new pods to become ready via a readiness check before scaling down the old components. If a significant issue occurs, the rolling deployment can be aborted.

We want to deploy application in production environment without any downtime to the client. If you want to implement CI/CD in production then you have to use this type of strategy so that you can continuous deliver on the production without any downtime to the client, currently what happened if we wants to deploy anything in the production we have to talk to client and ask for maintenance window (some downtime in production to upgrade the version during which client is not able to access the application) using RollingUpdate strategy there is no downtime , we are able to upgrade our application from version1 to version2 Pause RollingUpdate (In case of something fail we cannot proceed further, rolling back the pods)



`while true; do curl http://serviceIP; sleep 1; done;`

Kubernetes always keeps meta-data of all version.

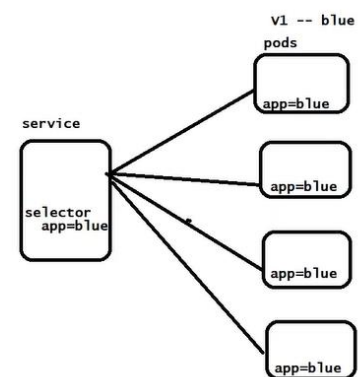
watch kubectl get rs

`// monitoring live status`

Blue/Green Update:

release window/maintenance window (sat 12am-6am)
blue-green deployment model

blue -- v1
green -- v2

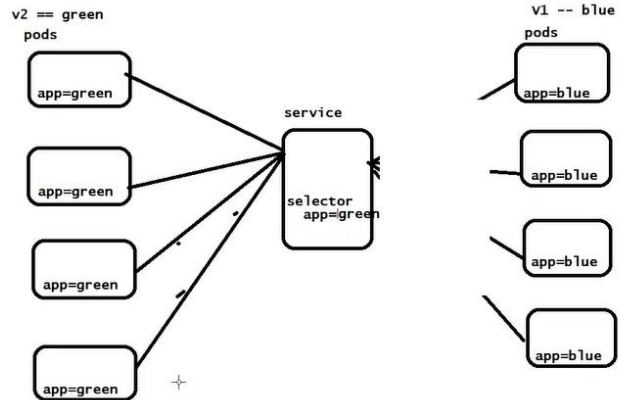


```

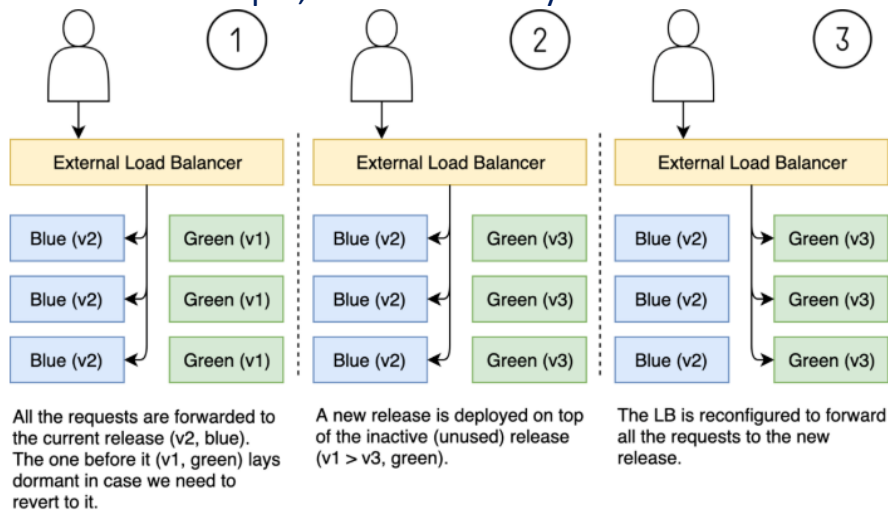
release window/maintenance window ( sat 12am-6am )
blue-green deployment model

blue -- v1
green -- v2

```



Blue-green deployment is a technique that reduces downtime and risk by running two identical production environments called Blue and Green. At any time, only one of the environments is live, with the live environment serving all production traffic. For this example, Blue is currently live and Green is idle.



We come with completely new setup of infrastructure every time even for small change,

- Blue represents current infrastructure.
- Green represents latest upgraded infrastructure.
- Just change the routing of Load Balancer from blue to green.

In case of any issue in new infrastructure just change the routing of LB to previous infrastructure.

```

kind: Deployment
apiVersion: apps/v1
metadata:
  name: blue-v1-deploy
spec:
  replicas: 5
  selector:
    matchLabels:
      app: blue
  template:
    metadata:
      labels:
        app: blue
    spec:
      containers:
        - name: app
          image: leaddevops/kuberserve:v1
~
~
~

```

while true; do curl <IP>:80; sleep 1; done

How you update the deployment?



```

> kubectl apply -f deployment-definition.yml
deployment "myapp-deployment" configured

> kubectl set image deployment/myapp-deployment \
  nginx=nginx:1.9.1
deployment "myapp-deployment" image is updated

```

```

deployment-definition.yml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.7.1
  replicas: 3
  selector:
    matchLabels:
      type: front-end

```

kubectl apply -f <definition>.yaml

When a new deployment is created, say to deploy 5 replicas, it first creates a Replicaset automatically, which in turn creates the number of PODs required to meet the number of replicas. When you upgrade your application as we saw in the previous slide, the kubernetes deployment object creates a NEW ReplicaSet under the hoods and starts deploying the containers there. At the same time

taking down the PODs in the old replica-set following a RollingUpdate strategy. This can be seen when you try to list the replicasets using the `kubectl get replicasets` command. Here we see the old ReplicaSet with 0 PODs and the new ReplicaSet with 5 PODs.

Say for instance once you upgrade your application, you realized something is wrong with new version you deployed, and you have to rollback your changes K8s rollout allow you to roll back to your previous version.

Autoscaling: On certain events, we want to increase/decrease number of pods

step1: deploy metrics server

`kubectl top nodes (CPU, Memory)`

`kubectl top pods (CPU, Memory)`

Horizontal Pod Scalar HPA

`k autoscale deploy <name> --min=2 --max=10 --cpu-percent=80`

`kubectl get hpa`

How to put load on the application?

step2: Install apache benchmark (Load Runner, JMeter)

`apt install apache2-utils`

`ab # apache bench`

Apache Bench (ab) is a load testing and bench marking tool for Hyper Transfer Protocol (HTTP) server. It can be run from command line and it is very simple to use. A quick load testing output can be obtained in just one minute.

step3: Get the service IP address using

`kubectl get svc <name of service>`

`curl <ip>: port`

`kubectl top nodes // metrics server by default shows only cpu and memory`

`kubectl top pods`

step4: To put the load use apache benchmark tool

`ab -n 50000 -c 1000 http://<svc ip>/`

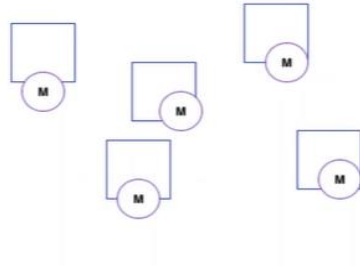
`/*`

n: number of requests

c: concurrent requests

metrics server provides only cpu and memory, if you want to customized can write your own script. */
hpa talks to metrics server to get the metrics

DaemonSet Controller (runs on each and every node, No scaling for DaemonSet):



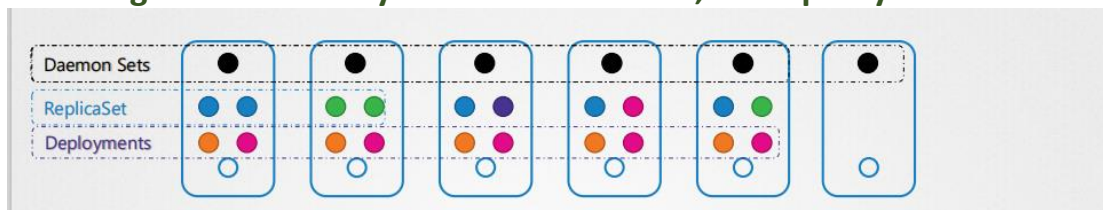
- No replicas, one pod per node
- DaemonSet Controller creates number of pods=number of nodes in the cluster.
- DaemonSet Controller will ensure that every node in the cluster have 1 pod only.
- DaemonSet Controller can't be scale manually (like kubectl scale)
- DaemonSet Controller scales up automatically when we add new node in cluster.
- DaemonSet Controller scales down automatically when we delete node from cluster.

UC:

- monitoring agent
- logging agents (splunk forwarder, ELK (fluentd))
- anti-virus
- kube-proxy
- kubelet

`kubectl get pods -n kube-system`

`kubectl get ds -n kube-system #weave-net, kube-proxy`



How to deploy **only one Pod on every node?**

Use Case: We have a requirement of monitoring app running on every node of the cluster. It ensures that all nodes of the cluster run copy of the Pod.

A DaemonSet ensures that all (or some) Nodes run a copy of a Pod. As nodes are added to the cluster, Pods are added to them. As nodes are removed from the cluster, those Pods are garbage collected. Deleting a DaemonSet will clean up the Pods it created.

Some typical uses of a DaemonSet are: running a logs collection daemon on every node, such as fluentd or filebeat. Running a node monitoring daemon on every node, such as Prometheus Node Exporter, Dynatrace OneAgent, AppDynamics Agent, Datadog agent.

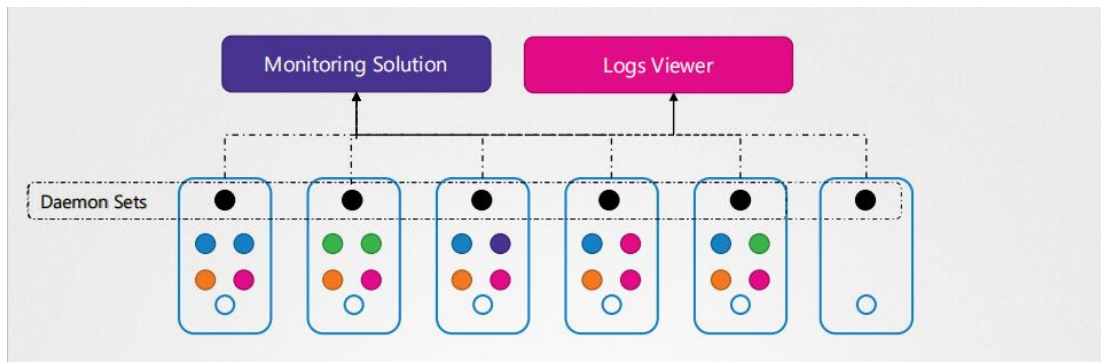
We don't define **replicas** parameter

Can't Scale, it ensures the just only one pod run on every node of the cluster.

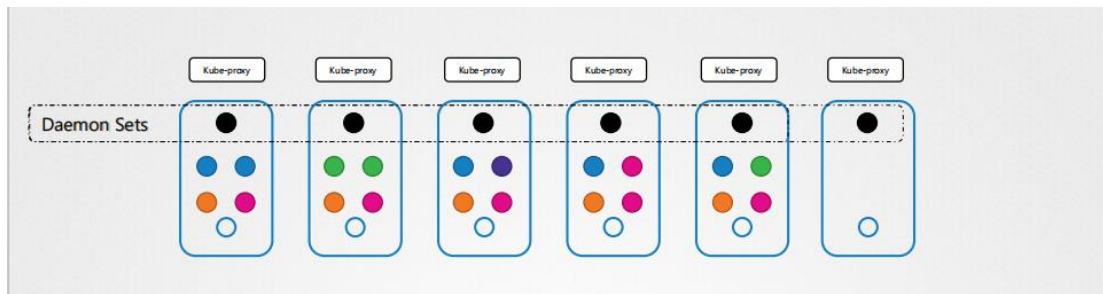
What is the purpose of this type of controller?

Something that run on the back ground e.g. monitoring agents (prometheus), to collect the metrics, logging of that particular server.

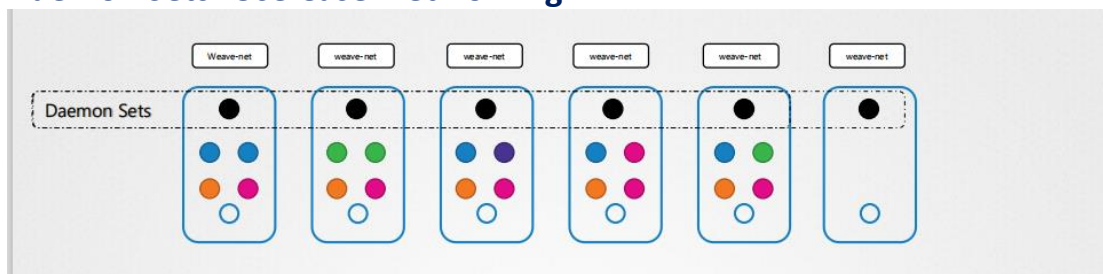
Daemon Sets- Use Case



Daemon Sets- Use Case-kube-proxy



Daemon Sets- Use Case-Networking



daemon-set-definition.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
        - name: monitoring-agent
          image: monitoring-agent
```

replicaset-definition.yaml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
        - name: monitoring-agent
          image: monitoring-agent
```

```
➤ kubectl create -f daemon-set-definition.yaml
```

```
daemon-set Created
```

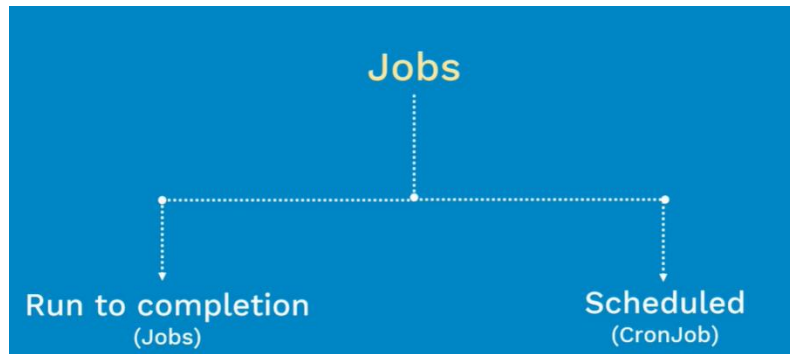
Job (Meant for completion)

- One time activity.
- Run for completion

UC: How do you run Pods till task completion?

Repeating a task at specific time?

Run-to-Completion (Jobs) - batch processing



Job: meant for completion for intended to do.

backoffLimit: how many numbers of times you retry in **case of failure**.

completions: # number of times

parallelism: # number of parallel pods

activeDeadlineSeconds: # once job reaches, all running pods terminated

If we delete the pod then rs will create it, but if we delete the container inside the pod who will create it? **Pod** itself because it has a default restartPolicy=Always (restart=Always)

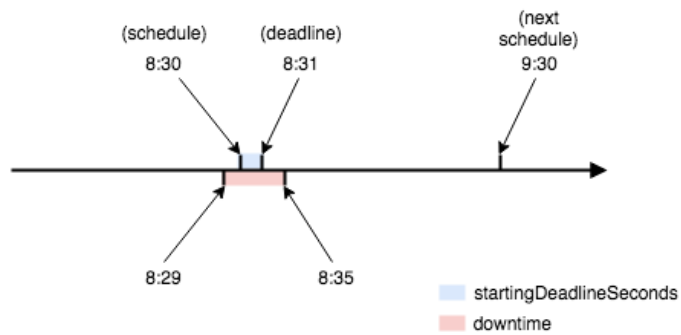
In Job **restart=Never** in **case of success**

CronJob: (batch processing, schedule)

The CronJob is only responsible for creating Jobs that match its schedule, and the Job in turn is responsible for the management of the Pods it represents.

startingDeadlineSeconds #

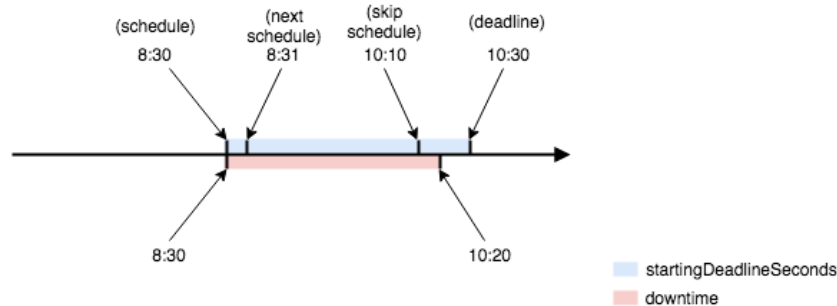
startingDeadlineSeconds means that if a CronJob controller cannot start a job run on its schedule, it will keep retrying until startingDeadlineSeconds is reached.



The interval of schedules > startingDeadlineSeconds

Example #1. Assume that the schedule is 8:30, 9:30, ... and `startingDeadlineSeconds` is 60 seconds. During 8:29-8:35, there is downtime to start a job run. In this case, the job will not be executed because when the system comes back to healthy, the time is 8:35 that is after the deadline 8:31. The schedule at 8:30 is lost and the next schedule will be at 9:30.

This is the case when the interval between schedules is greater than the `startingDeadlineSeconds`. What if it is less?



Example #2. Assume that the schedule is 8:30, 8:31, 8:32, ..., for every minute and `startingDeadlineSeconds` is 2 hours. The downtime is between 8:30 and 10:20.

What will happen in this case? The CronJob controller will keep trying to start a job according to the most recent schedule. Once the number of schedule misses reaches 100 times, the controller will not try to start a job anymore and logs the error:

CronJob creates a Job, and Job creates a Pod

Labels, Selector and Annotations

label: k8s related information

annotation: Third party objects

Labels and Selectors are a standard method to group things together.

Say you have a set of different species. A user wants to be able to filter them based on different criteria. Such as based on their class or based on their type – domestic or wild or say by color and not just group, you want to be able to filter them based on a criterion. Such as all green animals. **Labels are properties attached to each item. So, you add properties to each item for their class, kind and color.**

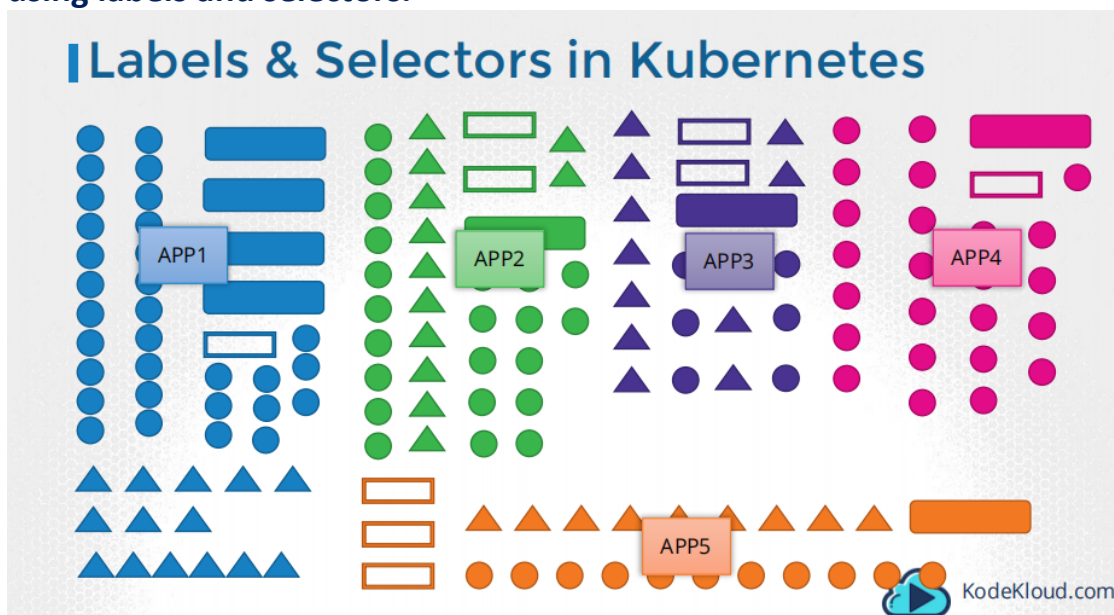
Selectors help you filter these items. For example, when you say class equals mammal, we get a list of mammals, and when you say color equals green, we get the green mammals.

We see labels and selectors used everywhere, such as the keywords you tag to youtube videos or blogs that help users filter and find the right content.

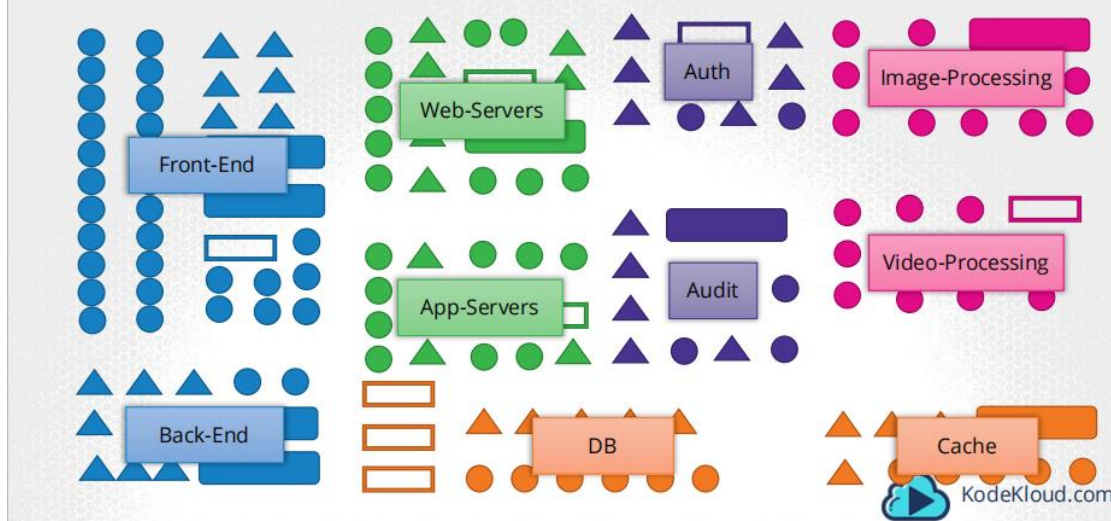


So how are labels and selectors used in Kubernetes?

We have created a lot of different types of Objects in Kubernetes. Pods, Services, ReplicaSet and Deployments. For Kubernetes, all of these are different objects. Over time you may end up having 100s and 1000s of these objects in your cluster. Then you will need a way to filter and view different objects by different categories. Such as to group objects by their type or view objects by application or by their functionality. Whatever it may be, you can group and select objects using labels and selectors.

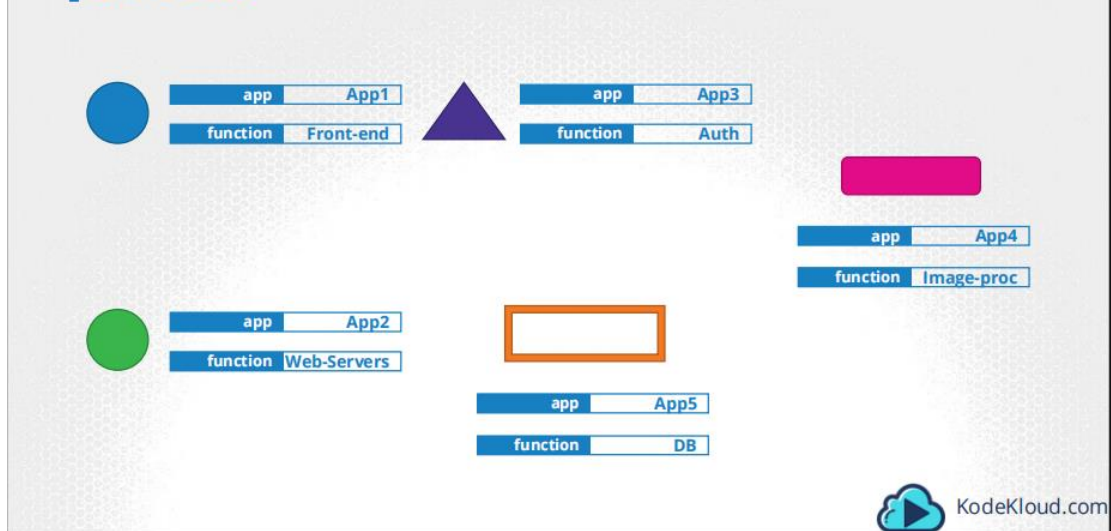


| Labels & Selectors in Kubernetes



For each object attach labels as per your needs, like app, function etc.

| Labels



Then while selecting, specify a condition to filter specific objects. For example, `app =App1`.

```
kubectl get pods --selector app=App1
```

```
kubectl get all --selector app=App1
```

```
kubectl get pods --selector env=prod, bu=finance, tier=frontend
```

Kubernetes uses labels and selectors to connect different type of objects.

Annotations are used to record, other records for informative purpose, third party e.g., docker information

|Annotations

replicaset-definition.yaml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: simple-webapp
  labels:
    app: App1
    function: Front-end
  annotations:
    buildversion: 1.34
spec:
  replicas: 3
  selector:
    matchLabels:
      app: App1
  template:
    metadata:
      labels:
        app: App1
        function: Front-end
    spec:
      containers:
        - name: simple-webapp
          image: simple-webapp
```

om